

List Of Mini Projects

1) Project Title Early Prediction of Sepsis Using Clinical Data

Abstract

Sepsis is a life-threatening medical condition caused by the body's extreme response to infection. Early detection is crucial for patient survival. This project uses machine learning models trained on clinical data (such as temperature, heart rate, blood pressure, and lab reports) to predict the onset of sepsis at an early stage. The system processes patient data, trains a classifier (e.g., Logistic Regression, Random Forest, or Neural Network), and outputs the probability of sepsis. The project aims to assist healthcare professionals in decision-making, reducing mortality rates.

Hardware Requirements

- Standard PC/Laptop (Intel i5 or higher, 8GB RAM, 500GB HDD/SSD)
- Optional: GPU (for faster ML training)

Software Requirements

- **Frontend:** Jupyter Notebook (Visualization of results, Matplotlib, Seaborn)
- **Backend:** Python 3.6.9 with ML libraries (scikit-learn, NumPy, Pandas)
- **Environment:** Anaconda Navigator, Jupyter Notebook
- **Dataset:** Clinical health records (training & testing datasets in .csv format)

2) Project Title Application-Based Portrait Building System

Abstract

Identifying suspects in criminal investigations often depends on vague eyewitness descriptions, which makes it difficult for law enforcement agencies to catch criminals. The **Portrait Building System (PBS)** is designed to assist Investigating Officers by creating digital facial composites of suspects. The system allows users to select and combine various facial components (head shape, eyes, nose, mouth, chin, etc.) from a digitized database, based on witness descriptions. The final portrait can be printed or shared digitally for circulation among police and media. This

List Of Mini Projects

project leverages image processing and database management techniques to streamline criminal identification, providing faster and more accurate suspect visualization.

Hardware Requirements

- Standard PC/Laptop with minimum Intel i5 processor, 8 GB RAM
- High-resolution monitor for image rendering
- Printer (optional, for physical copies of portraits)

Software Requirements

- **Frontend:**
 - Web/GUI interface (Java Swing / Python Tkinter / Web app with React or Angular)
 - Image rendering libraries (OpenCV, PIL in Python)
- **Backend:**
 - Database (MySQL / SQLite) to store digitized facial features
 - Python / Java for logic implementation
 - Image processing (OpenCV, TensorFlow/Keras for optional enhancements)
- **Environment:**
 - IDE: PyCharm, Jupyter Notebook, or Eclipse
 - OS: Windows/Linux

3) Project Title

Identification of Criminals with Respect to Inter/Intra Country Crimes

Abstract

List Of Mini Projects

With the increasing rate of theft, burglary, kidnapping, and human trafficking in urban areas, it has become essential for law enforcement agencies to identify whether the criminals belong to the same state, country, or from outside. Traditional methods of analyzing crime patterns require extensive manual effort from crime analysts. This project applies **machine learning algorithms** on historical crime datasets to detect hidden patterns in criminal behavior. The system is trained on large-scale records to predict whether an accused is an **intra-state, inter-state, or inter-country criminal**, providing actionable insights for faster investigations. By leveraging predictive analytics, the model enhances accuracy in identifying cross-border criminals and assists police in prioritizing actions.

Hardware Requirements

- Standard PC/Laptop with i5/i7 processor, 8–16 GB RAM
- Storage: 500 GB HDD/SSD
- Optional: GPU for faster ML training

Software Requirements

- **Frontend:**
 - Jupyter Notebook (for visualization & ML training results)
 - Web Dashboard (Flask/Django + Bootstrap or React) to display predictions
- **Backend:**
 - Python with ML libraries (scikit-learn, Pandas, NumPy, TensorFlow/Keras)
 - Database (MySQL / MongoDB for crime records)
 - Data preprocessing & analytics tools
- **Environment:**
 - Anaconda Navigator
 - Jupyter Notebook / PyCharm IDE
 - OS: Windows/Linux

List Of Mini Projects

4) Project Title Crime Management System

Abstract

The **Crime Management System (CMS)** is a web-based solution designed to assist law enforcement authorities in managing, monitoring, and analyzing crime data. It enables police officers to record First Information Reports (FIRs), track case progress, maintain criminal records, and generate analytical reports. Citizens can also use the system to report incidents online, making the process more transparent and efficient. The project ensures quick response times, improves communication between departments, and provides dashboards for visualizing crime trends across regions.

Hardware Requirements

- Standard PC/Laptop with Intel i5 processor, 8 GB RAM, 500 GB HDD/SSD
- Stable internet connection
- Optional: Server hosting for deployment (Cloud / On-premise)

Software Requirements

- **Frontend:**
 - Angular Framework (TypeScript, HTML, CSS, Bootstrap)
 - Ionic Framework (for hybrid mobile app interface)
- **Backend:**
 - Node.js with Express.js (for API handling)
 - Database: MySQL / MongoDB
 - Authentication using JWT (JSON Web Tokens)
- **Environment & Tools:**
 - Visual Studio Code (IDE)
 - Angular CLI & Ionic CLI

List Of Mini Projects

- npm (Node Package Manager)
- OS: Windows/Linux/macOS

5) Project Title Sentiment Analysis on Text Feedback

Abstract

Customer and user feedback plays a critical role in improving products and services. However, analyzing large amounts of textual feedback manually is time-consuming and prone to errors. This project uses **Natural Language Processing (NLP)** techniques and **Machine Learning models** to classify text feedback into positive, negative, or neutral sentiments. The model is trained on a labeled dataset of user reviews to detect patterns in language and emotion. Businesses and organizations can use this project to automatically analyze customer opinions, monitor public sentiment, and make data-driven decisions.

Hardware Requirements

- Standard PC/Laptop (Intel i5 or above, 8GB RAM, 500GB HDD/SSD)
- Optional: GPU for faster ML training

Software Requirements

- **Frontend:**
 - Jupyter Notebook (for training, testing, and visualization)
 - Matplotlib / Seaborn (for graphs & sentiment distribution visualization)
- **Backend:**
 - Python 3.6.9
 - NLP libraries (NLTK, TextBlob, Scikit-learn, Pandas, NumPy)
 - Dataset: CSV file with labeled text feedback
- **Environment:**

List Of Mini Projects

- Anaconda Navigator
- Jupyter Notebook IDE
- OS: Windows/Linux

6) **Project Title** Health Guide Monitoring App for the Disabled

Abstract

People with disabilities often face challenges in monitoring their health on a daily basis. This project aims to build an **IoT-enabled health guide monitoring application** that helps disabled individuals track their vital signs such as heart rate, body temperature, and oxygen levels. The system uses **Arduino microcontroller and ESP8266 Wi-Fi module** to collect sensor data and transmit it wirelessly. The data is displayed on a web-based dashboard accessible via PC or mobile devices, refreshing automatically every 15 seconds. This solution provides real-time health monitoring, improves accessibility for disabled users, and assists caregivers in taking timely action.

Hardware Requirements

- Arduino board (Arduino Uno/Nano)
- ESP8266 Wi-Fi Module
- Sensors (Heart Rate Sensor, Temperature Sensor, Pulse Oximeter module)
- Breadboard, Jumper Wires, Power Supply
- Laptop/PC/Mobile for monitoring

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS)
 - Auto-refresh mechanism for real-time health updates
- **Backend:**

List Of Mini Projects

- Arduino IDE (C/C++ for coding)
- ESP8266 Wi-Fi module firmware for communication
- Data hosted locally (device hotspot)
- **Environment:**
 - Arduino IDE for code writing & uploading
 - Browser-based interface (Chrome/Firefox)
 - OS: Windows/Linux

7 Project Title

One Nation, One Citizen, One Pension

Abstract

The government provides multiple pension schemes to ensure financial security for senior citizens and retired employees. However, managing different schemes across multiple banks and institutions often creates complexity and redundancy. This project aims to develop a **centralized pension management system** where every citizen is uniquely identified using their Aadhaar number and can apply for suitable pension schemes. Users register and submit requests online, which are verified by banks and approved or rejected by an admin. The system ensures transparency, reduces fraudulent claims, and provides citizens with easy access to different pension schemes under a single platform.

Hardware Requirements

- PC/Laptop with Intel i5 processor, 8GB RAM, 500GB HDD/SSD
- Stable internet connection
- Server setup (Tomcat on local machine or cloud deployment)

Software Requirements

- **Frontend:**

List Of Mini Projects

- JSP (Java Server Pages), HTML, CSS
- User interface for login/signup, scheme selection, and application form
- **Backend:**
 - Java (JDK 8) for business logic
 - Apache Tomcat Server (v7/v8) for deployment
 - MySQL Database for storing user details, pension scheme data, and authentication
- **Environment & Tools:**
 - Java Runtime Environment (JRE 8)
 - MySQL Workbench & Server
 - Apache Tomcat Server
 - IDE: Eclipse / NetBeans
 - OS: Windows/Linux

8) Project Title

Agritech: Stubble Aggregation and Disposal

Abstract

Stubble burning has become a major environmental concern, causing air pollution and loss of soil fertility. To address this problem, this project proposes a **web-based Agritech solution** for efficient **stubble aggregation and disposal**. The system connects **farmers, transporters, and industries** on a unified platform. Farmers can request stubble collection services, transporters can offer logistics, and industries can place requests for stubble-based raw materials. The backend uses **MySQL** for data management, while **Spring and Hibernate** provide business logic and database connectivity. The project promotes sustainable agriculture, reduces stubble burning, and encourages recycling through eco-friendly disposal.

Hardware Requirements

- PC/Laptop (Intel i5 or higher, 8GB RAM, 500GB HDD/SSD)

List Of Mini Projects

- Internet connection
- Optional: Server hosting (Tomcat deployment on cloud/on-premise)

Software Requirements

- **Frontend:**
 - HTML, CSS, JavaScript for web interface
 - Responsive web design for farmers, transporters, and industries
- **Backend:**
 - Java (Spring Framework for application development)
 - Hibernate ORM for connecting Java objects with MySQL database
 - MySQL database for storing user, transporter, and industry data
 - Apache Tomcat Server for deployment
- **Environment & Tools:**
 - Eclipse IDE for Java EE Developers
 - MySQL Workbench
 - Tomcat Server (local deployment)
 - OS: Windows/Linux

9) Project Title

AI-Based Traffic Management System for Real-Time Traffic Control

Abstract

Traffic congestion is one of the most pressing problems in modern cities, leading to wasted time, increased fuel consumption, and high environmental pollution. Traditional traffic light systems operate on static timing, which is not efficient in handling fluctuating traffic conditions. This project proposes an **AI-powered real-time traffic management system** that integrates

List Of Mini Projects

IoT sensors and surveillance cameras to monitor traffic density, vehicle speed, and congestion levels. Using **machine learning algorithms**, the system dynamically adjusts traffic light timings, prioritizing heavily congested lanes to ensure smooth traffic flow. The solution aims to minimize waiting times, reduce accidents, and improve overall urban mobility.

Hardware Requirements

- IoT sensors (IR/Ultrasonic sensors for vehicle detection)
- Surveillance cameras (CCTV with computer vision support)
- Raspberry Pi / Arduino for sensor integration
- Standard PC/Laptop with GPU support (for AI model processing)

Software Requirements

- **Frontend:**
 - Web dashboard (HTML, CSS, JavaScript, React/Angular) for traffic visualization
 - Real-time traffic maps with congestion indicators
- **Backend:**
 - Python (OpenCV for video processing, TensorFlow/Keras for AI model)
 - Machine Learning models (for vehicle detection & adaptive signal timing)
 - Database: MySQL/MongoDB to store traffic logs
 - Flask/Django server for backend logic
- **Environment & Tools:**
 - Anaconda (for ML model training)
 - Jupyter Notebook (for development & testing)
 - Raspberry Pi/Arduino IDE (for IoT integration)
 - OS: Windows/Linux

List Of Mini Projects

10) Project Title

AI-Enhanced Career Job Matching System

Abstract

Job seekers often struggle to identify roles that match their skills, while employers face difficulties in shortlisting candidates efficiently. This project presents an **AI-powered career job matching system** that automatically extracts skills from resumes and compares them with job descriptions to calculate a **similarity score**. Using **GloVe embeddings** for semantic understanding and **cosine similarity** for matching, the system provides accurate real-time analysis of candidate-job fit. The web-based platform displays match percentages and ranks suitable jobs, helping users identify the best opportunities and enabling employers to shortlist candidates effectively.

Hardware Requirements

- Standard PC/Laptop (Intel i5 or above, 8–16 GB RAM)
- Optional GPU for faster NLP computation

Software Requirements

- **Frontend:**
 - React.js for interactive UI
 - HTML, CSS, JavaScript for visualization
 - Dashboard showing matched jobs with similarity percentages
- **Backend:**
 - Python with Flask framework (for API and backend logic)
 - NLP Libraries: Gensim, spaCy, NLTK
 - GloVe embeddings for semantic understanding
 - Cosine similarity computation for skill-job matching
 - Database: MySQL / MongoDB to store resumes and job postings

List Of Mini Projects

- **Environment & Tools:**

- Anaconda for Python environment management
- Jupyter Notebook for model development & testing
- OS: Windows/Linux

11) Project Title

AI-Driven Research Engine for Commercial Courts

Abstract

Legal research in commercial courts is often time-consuming due to the volume of case laws, statutes, and regulations. This project proposes an **AI-driven research engine** that aggregates legal data, extracts relevant information, and provides customized results to expedite case analysis. Using **predictive analytics** on historical case outcomes, the system forecasts possible results and supports lawyers, judges, and legal professionals in faster decision-making. The platform ensures **data localization**, multilingual support, and a user-friendly interface while adhering to ethical AI principles. The system acts as a **neutral facilitator**, helping improve the ease of doing business by reducing delays in commercial dispute resolution.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7, 8–16 GB RAM)
- Optional GPU for AI model processing
- Stable internet connection for data aggregation

Software Requirements

- **Frontend:**
 - Web interface using HTML, CSS, JavaScript (React.js/Angular optional)
 - Dashboard displaying case summaries, search results, and predictions
- **Backend:**

List Of Mini Projects

- Python (Flask/Django for API development)
- NLP libraries (spaCy, NLTK, Gensim) for legal text processing
- Machine Learning libraries (Scikit-learn, TensorFlow/Keras) for predictive analytics
- Database: MySQL / MongoDB to store legal documents, case data, and metadata
- **Environment & Tools:**
 - Anaconda for Python environment
 - Jupyter Notebook for AI model development and testing
 - OS: Windows/Linux

12) Project Title

LLM-Based Verilog Code Generation for Digital Chip Design

Abstract

Designing and validating digital chips is a complex and time-consuming task, requiring specialized knowledge and repetitive workflows. This project leverages **Large Language Models (LLMs)** to automate the generation of **Verilog Register Transfer Level (RTL) code** directly from natural language prompts. By fine-tuning pre-trained LLMs using **PEFT frameworks** with **LoRA and 4-bit quantization**, the system efficiently adapts to the syntax and semantics of Verilog. The process accelerates chip design, reduces development cycles, and improves code quality, consistency, and reliability. The approach demonstrates that AI can bridge the gap between high-level human design concepts and low-level machine-interpretable code, making digital chip design more accessible and innovative.

Hardware Requirements

- Standard PC/Laptop with Intel i5/i7 or AMD Ryzen, 16 GB RAM (32 GB recommended)
- GPU-enabled machine (NVIDIA with CUDA support) for faster LLM training

List Of Mini Projects

- Optional: Cloud platforms (Kaggle, Google Colab)

Software Requirements

- **Frontend:**
 - Jupyter Notebook / VS Code interface for prompt input and code generation
 - Visualization of generated Verilog modules (optional using waveform viewers)
- **Backend:**
 - Python 3.x
 - Hugging Face Transformers / PEFT Framework
 - SFTTrainer library for fine-tuning LLM
 - Verilog simulator (ModelSim / Icarus Verilog) to test generated code
 - Dataset: Verilog RTL examples for training
- **Environment & Tools:**
 - Anaconda for Python environment
 - Google Colab or Kaggle for cloud-based training
 - OS: Windows/Linux

13) Project Title

Timeless Traditions: Exploring Indian Culture and Heritage

Abstract

India's vast cultural heritage offers a wealth of opportunities for education, tourism, and global awareness. This project aims to create **immersive virtual experiences** of India's landmarks, art forms, music, and festivals, enabling users to explore the nation's traditions remotely. Additionally, it promotes rural tourism, showcasing traditional crafts, dances, and local cuisines

List Of Mini Projects

to provide authentic cultural experiences. The project also integrates **e-commerce platforms** to support artisans by displaying handcrafted goods, textiles, and jewelry to a global audience. Through virtual tours, interactive workshops, cultural festivals, and online marketplaces, the system preserves and promotes India's heritage while making it accessible to contemporary audiences worldwide.

Hardware Requirements

- Standard PC/Laptop (Intel i5 or higher, 8–16 GB RAM)
- Internet connection for accessing virtual experiences and e-commerce features
- Optional: VR headset for immersive virtual reality tours

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js/Angular optional)
 - Virtual tour visualization using WebGL / Three.js for 3D rendering
- **Backend:**
 - Python / Node.js for server-side logic
 - Database: MySQL / MongoDB for storing artisan data, cultural content, and user interactions
 - APIs for integrating VR content and e-commerce functionalities
- **Environment & Tools:**
 - VS Code or WebStorm IDE
 - Blender / Unity (optional) for 3D content creation
 - OS: Windows/Linux

14) Project Title

Face Mask Detection System

List Of Mini Projects

Abstract

The Face Mask Detection System is designed to automatically identify whether individuals are wearing face masks in real-time, ensuring compliance with public health protocols. Using **computer vision and machine learning**, particularly **Convolutional Neural Networks (CNNs)**, the system processes images or video streams to detect masked and unmasked faces. The project involves collecting a large dataset of masked and unmasked faces, training a CNN model, and deploying it to classify faces in real-time video feeds. This system is particularly useful in public spaces, workplaces, and high-traffic areas to reduce virus transmission and enhance safety measures.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Webcam or IP camera for real-time video feed
- Optional: GPU (NVIDIA) for faster model training

Software Requirements

- **Frontend:**
 - Python GUI (Tkinter / PyQt) or web dashboard (Flask/Django + HTML/CSS/JS)
 - Visualization of detection results on live video feed
- **Backend:**
 - Python 3.x
 - Machine Learning Libraries: TensorFlow / Keras / PyTorch
 - OpenCV for real-time image/video processing
 - Dataset: Masked and unmasked face images
- **Environment & Tools:**
 - Anaconda for environment management
 - Jupyter Notebook / VS Code for development

List Of Mini Projects

- OS: Windows/Linux

15) Project Title

Fit Genie: AI-Powered Virtual Fitness Coach

Abstract

Fit Genie is an **AI-powered virtual fitness coach** designed to provide personalized workout and nutrition plans based on individual fitness goals. Users input their objectives, fitness level, available equipment, and time constraints. The AI then generates adaptive workout routines and meal plans tailored to the user's preferences, such as vegetarian, gluten-free, or other diets. Integration with fitness wearables (Fitbit, Apple Watch) allows real-time tracking of steps, heart rate, and calories burned. The system continuously updates recommendations to optimize progress, providing a holistic solution for health, wellness, and goal-oriented fitness management.

Hardware Requirements

- PC/Laptop or Smartphone for app access
- Fitness wearables (Fitbit, Apple Watch, or other trackers)
- Optional: GPU-enabled system for AI training

Software Requirements

- **Frontend:**
 - Mobile app interface (Flutter / React Native / Android Studio / Swift)
 - Web dashboard (HTML, CSS, JavaScript) for data visualization
 - Real-time updates from wearable devices
- **Backend:**
 - Python / Node.js for server-side processing
 - AI/ML Libraries: TensorFlow / Keras / Scikit-learn for personalized plan generation

List Of Mini Projects

- Database: MySQL / MongoDB for user profiles, workout, and nutrition data
- APIs for wearable integration (Fitbit API, Apple HealthKit)
- **Environment & Tools:**
 - Anaconda for AI/ML environment
 - VS Code / Android Studio / Xcode for development
 - OS: Windows/Linux/macOS

16) Project Title

FlexiLearn: Empowering Education in the Digital Age

Abstract

FlexiLearn is a smart education platform that leverages AI, data analytics, and interactive tools to provide a personalized and adaptive learning experience. The system enables learners to study flexibly, receive real-time feedback, and access tailored content at their own pace. By analyzing user performance and engagement, AI recommends optimized learning paths, helping students achieve their goals efficiently. The platform supports remote access, making education accessible anytime, anywhere, and fosters a dynamic, engaging, and student-centered environment. FlexiLearn represents a forward-looking educational model that adapts to the needs of modern learners in the digital age.

Hardware Requirements

- Standard PC/Laptop or tablet/smartphone for learners
- Internet connection for real-time learning and updates
- Optional: Server hosting for AI analytics and backend processing

Software Requirements

List Of Mini Projects

- **Frontend:**

- Web interface (HTML, CSS, JavaScript, React.js/Angular)
- Mobile app interface (Flutter / React Native / Android Studio)
- Interactive learning tools (quizzes, videos, simulations)

- **Backend:**

- Python / Node.js for server-side logic
- AI/ML Libraries: TensorFlow / Keras / Scikit-learn for adaptive learning and recommendations
- Database: MySQL / MongoDB for storing user profiles, course content, and performance data
- APIs for real-time analytics and content delivery

- **Environment & Tools:**

- Anaconda for AI/ML environment
- VS Code / Android Studio / Xcode for development
- OS: Windows/Linux/macOS

17)Project Title

Freelancing Platform: AI-Enhanced Digital Talent Management

Abstract

The proposed freelancing platform is designed to streamline **digital talent acquisition and project management** in the growing gig economy. The system uses **AI-driven matching algorithms** and skill assessment frameworks to connect businesses with qualified freelancers across multiple domains such as technology, design, writing, and marketing. Features include

List Of Mini Projects

dynamic rating mechanisms, secure payment processing, and real-time collaboration tools to ensure a reliable and professional ecosystem. Additionally, the platform offers personalized project recommendations, intelligent contract management, and scalable architecture to optimize the project lifecycle from posting to delivery. This solution enhances both freelancer and client experiences, ensuring efficiency, transparency, and high-quality project outcomes.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7, 8–16 GB RAM)
- Stable internet connection
- Optional: Cloud server for deployment and scalability

Software Requirements

- **Frontend:**
 - HTML, CSS, JavaScript (React.js / Angular)
 - Responsive UI for freelancer and client dashboards
 - Real-time collaboration interface (chat, project updates)
- **Backend:**
 - Node.js / Django / Flask for server-side logic
 - AI/ML Libraries: TensorFlow / Scikit-learn for job-freelancer matching and recommendations
 - Database: MySQL / MongoDB for user profiles, project data, and transactions
 - Payment gateway integration (Stripe, PayPal, Razorpay)
- **Environment & Tools:**
 - VS Code / WebStorm for development
 - Git for version control
 - OS: Windows/Linux

List Of Mini Projects

18) Project Title

Groundwater Explorer: AI Chatbot for Ground Water Resource Insights

Abstract

Groundwater management is critical for sustainable water usage in agriculture, industry, and residential needs. This project proposes an **AI-powered chatbot** that provides real-time insights into groundwater resources, including water levels, hydrogeological data, water quality, and relevant reports. By leveraging **AI and machine learning**, the system processes large datasets and real-time information to answer user queries accurately. The chatbot serves as a valuable tool for local authorities, farmers, and researchers by facilitating informed decision-making, promoting sustainable groundwater management, and improving access to critical water resource information.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7, 8–16 GB RAM)
- Internet connection for real-time data access
- Optional: Cloud hosting server for chatbot deployment

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js/Angular) for chatbot interaction
 - Optional mobile app interface for remote access
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - AI/ML Libraries: TensorFlow / Keras / Scikit-learn for NLP and question-answering
 - NLP tools: spaCy, NLTK, or Hugging Face Transformers for language understanding

List Of Mini Projects

- Database: MySQL / MongoDB for storing groundwater datasets and historical reports
- **Environment & Tools:**
 - Anaconda for Python environment
 - Jupyter Notebook for AI/ML model development
 - OS: Windows/Linux

19) Project Title

AI-Based Interactive Chatbot for Department of Justice Website

Abstract

The Department of Justice (DoJ) oversees critical judicial functions, including eCourts projects, legal aid services, fast-track courts, and Tele Law services. This project proposes an **AI-powered interactive chatbot** for the DoJ website to provide users with instant, accurate information about court operations, case statuses, legal procedures, and government schemes. The chatbot leverages **Natural Language Processing (NLP)** and machine learning to understand user queries, deliver relevant answers, and improve its responses over time. It can handle information such as judge appointments, current vacancies, case pendency from the National Judicial Data Grid (NJDG), traffic fine payments, eFiling/ePay processes, and streaming of live court cases. By integrating AI, the platform enhances citizen engagement, accessibility, and efficiency in obtaining legal information.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7, 8–16 GB RAM)
- Internet connection for real-time data fetching
- Optional: Cloud hosting server for large-scale deployment

Software Requirements

- **Frontend:**

List Of Mini Projects

- Web interface (HTML, CSS, JavaScript, React.js / Angular) for chatbot interaction
- Dashboard for administrators to manage queries and chatbot learning
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - NLP Libraries: spaCy, NLTK, or Hugging Face Transformers for query understanding
 - Machine Learning libraries: TensorFlow / PyTorch for AI model training
 - Database: MySQL / MongoDB for storing DoJ data, FAQs, and historical queries
 - APIs to fetch real-time information from NJDG, eCourts services, and Tele Law platforms
- **Environment & Tools:**
 - Anaconda for Python environment
 - Jupyter Notebook / VS Code for AI model development
 - OS: Windows/Linux

20) Project Title

Mental Health LLM: AI-Powered Early Detection System

Abstract

The rise of mental health challenges has created a need for innovative tools to support early detection and intervention. This project leverages **Large Language Models (LLMs)**, such as BERT or GPT, to analyze text for early signs of mental health issues like anxiety, depression, or other disorders. By examining language patterns, tone, sentiment, and word usage, the system identifies linguistic markers associated with mental health conditions. Data is collected, preprocessed (text cleaning, tokenization), and used to train supervised or unsupervised models for classification or clustering. Ethical considerations, including privacy, fairness, and non-harmful labeling, are emphasized. This system provides **automatic, real-time mental health**

List Of Mini Projects

screening, offering insights that support traditional assessments and making mental healthcare more accessible globally.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 16 GB RAM)
- Optional: GPU (NVIDIA) for faster LLM training and inference
- Internet connection for cloud-based model access or API integration

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js/Angular) for user input and report visualization
 - Optional: Mobile app interface for accessibility
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - AI/ML Libraries: TensorFlow / Keras / PyTorch for LLM model training
 - NLP Libraries: Hugging Face Transformers, spaCy, NLTK for text processing
 - Database: MySQL / MongoDB for storing anonymized text data and results
- **Environment & Tools:**
 - Anaconda for Python environment management
 - Jupyter Notebook / VS Code for model development
 - OS: Windows/Linux

21) Project Title

Online Chatbot-Based Ticketing System for Museums

List Of Mini Projects

Abstract

This project proposes a **multilingual chatbot-based ticketing system** for museums to streamline ticket bookings, enhance visitor experience, and improve operational efficiency. The system handles bookings for general entry, exhibitions, and special shows, integrating a **secure payment gateway** for seamless transactions. Using **Natural Language Processing (NLP)** and machine learning, the chatbot understands visitor queries, provides accurate responses, and collects data on visitor preferences, peak times, and popular events. Analytics derived from this data can support **predictive crowd management, marketing strategies, and event planning**. By automating ticket booking, reducing human errors, and providing actionable insights, the system modernizes museum operations and elevates visitor engagement.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Internet connection for real-time bookings and chatbot access
- Optional: Cloud hosting server for scalability

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Chatbot interface for booking tickets
 - Dashboard for admin to view analytics
- **Backend:**
 - Python (Flask / Django) or Node.js for server-side logic
 - AI/ML Libraries: TensorFlow / PyTorch / Scikit-learn for NLP enhancement
 - NLP Libraries: spaCy, NLTK, or Hugging Face Transformers
 - Database: MySQL / MongoDB for storing visitor data, bookings, and analytics
 - Payment Gateway Integration: Stripe, PayPal, Razorpay

List Of Mini Projects

- **Environment & Tools:**

- Anaconda for Python environment management
- VS Code / WebStorm for development
- OS: Windows/Linux

22) Project Title

Plant Disease Detection Using Convolutional Neural Networks (CNN)

Abstract

Crop diseases pose a significant threat to agricultural productivity and food security. This project leverages **Convolutional Neural Networks (CNNs)** to automate the detection of plant diseases through image analysis. By extracting intricate visual features from leaf images, the CNN model can identify disease patterns accurately and in real-time. Farmers and agricultural experts can upload images of crops through a user-friendly interface to receive immediate predictions regarding plant health. This approach reduces dependence on manual inspection, facilitates early intervention, and enables cost-effective disease management, ultimately enhancing yield quality and ensuring sustainable agricultural practices.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- GPU (NVIDIA recommended) for faster CNN training and inference
- Internet connection for cloud deployment (optional)

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Upload module for leaf/crop images
 - Dashboard for disease prediction results

List Of Mini Projects

- **Backend:**
 - Python (Flask / Django) for server-side processing
 - AI/ML Libraries: TensorFlow / Keras / PyTorch for CNN model training
 - OpenCV for image preprocessing
 - Database: MySQL / MongoDB to store images, predictions, and user history
- **Environment & Tools:**
 - Anaconda for Python environment management
 - Jupyter Notebook / VS Code for model development
 - OS: Windows/Linux

23) Project Title

Heart Health Assessment and Feedback System

Abstract

This project presents an **accessible platform** for assessing and enhancing cardiovascular health awareness. Users input personal health details through a dedicated form, and the system employs **advanced algorithms** to categorize heart health risk levels into high, medium, or low. Personalized feedback is provided based on the risk category:

- **High risk:** Suggests consulting a cardiac specialist with hospital information and contact details.
- **Medium risk:** Provides dietary and exercise recommendations to reduce risk.
- **Low risk:** Encourages lifestyle documentation for continuous monitoring.

The system securely stores user data and tracks health trends over time. By integrating health technology and preventive care, the platform promotes a proactive approach to heart health management and supports the development of a health-conscious community.

Hardware Requirements

List Of Mini Projects

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Internet connection for web-based access
- Optional: Cloud server for scalable deployment

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Forms for health data input and dashboard for feedback visualization
- **Backend:**
 - Python (Flask / Django) or Node.js for server-side logic
 - AI/ML Libraries: Scikit-learn / TensorFlow / Keras for heart risk assessment algorithms
 - Database: MySQL / MongoDB for secure storage of health data
 - Optional: APIs for hospital information, dietary plans, or exercise recommendations
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for algorithm development
 - OS: Windows/Linux

24) Project Title

QuesGen: AI-Based Quiz Generator

Abstract

List Of Mini Projects

Creating quizzes manually from textbooks, research papers, or study materials is time-consuming for educators and may lead to incomplete coverage of essential topics. **QuesGen** addresses this by leveraging **AI** to automatically generate quiz questions from PDFs and other educational documents. The system analyzes content to extract key concepts and produces questions in multiple formats, including **multiple-choice, fill-in-the-blank, and true/false**, ensuring a comprehensive assessment of students' understanding. The AI model improves over time using feedback, producing higher-quality questions and adapting to a variety of subjects. This tool saves educators time, streamlines assessment creation, and enhances student learning through engaging and well-structured quizzes.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Internet connection for cloud processing (optional)
- Optional: GPU for faster NLP model inference

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Upload module for PDFs or other educational materials
 - Dashboard for viewing and editing generated quizzes
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - AI/ML Libraries: TensorFlow / PyTorch / Scikit-learn for NLP and question generation
 - NLP Libraries: Hugging Face Transformers, spaCy, NLTK for content analysis and summarization
 - Database: MySQL / MongoDB to store uploaded documents, generated quizzes, and feedback
- **Environment & Tools:**

List Of Mini Projects

- Anaconda for Python environment management
- Jupyter Notebook / VS Code for model development
- OS: Windows/Linux

25) Project Title

Retail Finance Advisor: AI-Based Financial Analysis for Retail Operations

Abstract

The project focuses on developing an **AI-powered financial advisory system** for retail store owners and branch managers. Leveraging **Large Language Models (LLMs)** like LLaMA3 and **Retrieval-Augmented Generation (RAG)**, the system analyzes data from multiple sources, including financial reports, sales data, and tax records. By integrating retrieval and generation mechanisms, it provides **personalized, contextually accurate insights** on key financial metrics such as gross income, cash flow, expenses, and taxes. The system empowers store owners to make data-driven decisions, gain a holistic view of branch performance, and optimize financial management across their retail operations.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 16 GB RAM)
- Optional: GPU (NVIDIA) for LLM inference
- Internet connection for accessing cloud APIs and financial data sources

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Dashboard for visualizing financial metrics and insights
- **Backend:**

List Of Mini Projects

- Python (Flask / Django) for server-side logic
- LLM Libraries: Hugging Face Transformers, LLaMA3 integration
- RAG implementation for retrieval and context-aware generation
- Database: MySQL / PostgreSQL / MongoDB for storing financial reports and sales data
- Optional: APIs to fetch live financial data
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for LLM and RAG model development
 - OS: Windows/Linux

26) Project Title

Simple and Effective Baseline for Text-to-Image Synthesis Using DF-GAN

Abstract

Text-to-image synthesis has gained significant attention with the advancement of **Generative Adversarial Networks (GANs)**, enabling automatic creation of high-quality images from textual descriptions. This project explores the **Deep Fusion GAN (DF-GAN)** architecture for efficient and coherent text-to-image generation. DF-GAN fuses textual and visual features through a **Deep Text-Image Fusion Block** and ensures semantic consistency using a **Target-Aware Discriminator**. By employing a one-stage generator, it overcomes issues like weak text-image alignment and computational inefficiencies while producing high-resolution, visually appealing images. The system is trained on diverse datasets and evaluated using metrics like **Fréchet Inception Distance (FID)** and **Inception Score (IS)**. This project has practical applications in **digital media, automated content creation, and interactive AI systems**, providing a strong baseline for future research in text-to-image synthesis.

Hardware Requirements

- High-performance PC/Laptop with GPU (NVIDIA RTX series recommended)

List Of Mini Projects

- Minimum 16 GB RAM
- Internet connection for dataset downloads and cloud GPU usage (optional)

Software Requirements

- **Backend / AI Model:**
 - Python (3.8 or above)
 - Deep Learning Libraries: PyTorch / TensorFlow
 - GAN-specific libraries: DF-GAN implementation packages
 - NLP Libraries: Hugging Face Transformers for text embedding
- **Frontend / Visualization:**
 - Web interface (HTML, CSS, JavaScript, Flask / Django)
 - Image display module for generated outputs
- **Database / Storage:**
 - MySQL / MongoDB for storing generated images and metadata (optional)
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for coding and training
 - OS: Windows/Linux

27)Project Title

Smart Agro-Advisor: AI-Based Fertilizer Recommendation System

Abstract

List Of Mini Projects

Excessive and improper fertilizer usage can degrade soil quality, reduce agricultural productivity, and lower farmers' incomes. **Smart Agro-Advisor** is an integrated application that provides **data-driven fertilizer recommendations** tailored to soil health, crop type, and local weather conditions. By leveraging **machine learning algorithms** and agricultural data analytics, the system analyzes soil nutrient levels, predicts crop requirements, and suggests optimal fertilizer types and quantities. Key features include **real-time weather integration, soil nutrient analysis, and predictive modeling** to enhance crop yield, minimize environmental impact, and support sustainable farming practices. This tool empowers farmers to make informed decisions, increase profitability, and maintain long-term soil health.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Internet-enabled device for real-time weather updates
- Optional: Soil sensors for real-time soil data acquisition

Software Requirements

- **Frontend:**
 - Web or mobile interface (HTML, CSS, JavaScript, React.js / Angular)
 - Dashboard for visualizing recommendations and soil data
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - AI/ML Libraries: Scikit-learn / TensorFlow / Keras for predictive modeling
 - Database: MySQL / MongoDB to store soil data, crop details, and recommendations
 - APIs: Weather API for real-time data integration
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for ML model development

List Of Mini Projects

- OS: Windows/Linux

28) Project Title

Transforming Text into SQL Queries Using NLP

Abstract

Interacting with databases often requires knowledge of SQL syntax, which can be a barrier for non-technical users. This project develops a **Text-to-SQL conversion system** that leverages **Natural Language Processing (NLP)** techniques to translate user-provided textual queries into accurate SQL commands. The system allows users to interact with databases naturally, enabling them to query, retrieve, and manipulate data without writing SQL code manually. By bridging the gap between human language and machine-readable SQL, the project empowers users to access and analyze data efficiently, making database interaction more intuitive and accessible.

Hardware Requirements

- Standard PC/Laptop (Intel i5/i7 or AMD Ryzen, 8–16 GB RAM)
- Internet connection for cloud APIs (optional)

Software Requirements

- **Frontend:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Input form for natural language queries
 - Display area for query results
- **Backend:**
 - Python (Flask / Django) for server-side logic
 - NLP Libraries: Hugging Face Transformers, spaCy, NLTK for query parsing
 - SQL Database: MySQL / PostgreSQL / SQLite for executing generated queries
 - Optional: Pretrained Text-to-SQL models like **SQLova** or **T5-based models**

List Of Mini Projects

- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for model development
 - OS: Windows/Linux

29)Project Title

Transformer-Based Polymer Property Prediction Model

Abstract

The project aims to develop a **chemically aware polymer tokenizer** using **transformer-based machine learning models** to predict polymer properties efficiently. Traditional computational methods like **Density Functional Theory (DFT)** and **Molecular Dynamics (MD)** simulations are time-consuming and computationally intensive. By leveraging transformers, the system encodes complex polymer chemical structures into representations that capture key chemical properties, enabling accurate **structure-property relationship analysis**. This approach accelerates polymer property prediction, reduces computational costs, and bridges the gap between computational and experimental methods. The model can be used to design polymers with desired properties, enhancing material innovation and research efficiency.

Hardware Requirements

- High-performance PC/Laptop with GPU (NVIDIA RTX series recommended)
- Minimum 16 GB RAM
- Optional: Cloud GPU resources (Google Colab, AWS, or Azure)

Software Requirements

- **Backend / Model Development:**
 - Python (3.8 or above)
 - Deep Learning Libraries: PyTorch / TensorFlow

List Of Mini Projects

- Transformers: Hugging Face Transformers for polymer sequence modeling
- RDKit / Chemprop for chemical feature extraction and preprocessing
- **Frontend / Visualization:**
 - Web interface (HTML, CSS, JavaScript, React.js / Angular)
 - Display module for predicted polymer properties
- **Database / Storage:**
 - MySQL / MongoDB to store polymer structures and predictions
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for model training and testing
 - OS: Windows/Linux

30) Project Title

Weapon Detection Alert System Using YOLOv8

Abstract

The Weapon Detection Alert System is designed to enhance **public safety** through **real-time monitoring** and automated alerts. Leveraging the **YOLOv8 deep learning model**, the system detects weapons in live CCTV feeds with high accuracy, overcoming challenges such as lighting variations, motion blur, and crowded scenes. Implemented as a **Flask-based web application**, it processes video streams instantaneously and triggers automated alerts with **timestamped images** to law enforcement for rapid response. Experimental results demonstrate improved detection accuracy and faster response times compared to traditional manual monitoring. Future enhancements may include **edge computing**, multi-camera synchronization, and expanded threat detection for comprehensive security coverage.

Hardware Requirements

- PC/Laptop with NVIDIA GPU (RTX series recommended) for real-time inference

List Of Mini Projects

- Minimum 16 GB RAM
- Internet-enabled device for notifications and alerts
- CCTV or video stream input device

Software Requirements

- **Backend / Model Development:**
 - Python (3.8 or above)
 - Deep Learning Libraries: PyTorch / OpenCV
 - YOLOv8 model for weapon detection
 - Flask for web application development
 - Libraries: NumPy, Pandas, OpenCV for video processing
- **Frontend / User Interface:**
 - HTML, CSS, JavaScript for dashboard display
 - Alert notification module (email/SMS integration optional)
- **Database / Storage:**
 - MySQL / MongoDB to store detection logs and captured frames
- **Environment & Tools:**
 - Anaconda for Python environment management
 - VS Code / Jupyter Notebook for model development
 - OS: Windows/Linux

31) Project Title

Direct Contract Farming System for Stable Market Access

List Of Mini Projects

Abstract

The Direct Contract Farming System provides a **structured framework** for farmers and buyers to formalize agreements, ensuring **stable market access** and **guaranteed pricing**. By reducing risks associated with price fluctuations and market uncertainties, the system benefits both farmers and buyers. Farmers gain income security, access to modern inputs, and technical support, while buyers secure a reliable supply of quality produce. Long-term partnerships foster **sustainable agricultural practices**, enhance supply chain efficiency, and contribute to rural economic growth. This system also supports national efforts toward **food security** and empowers smallholder farmers to thrive in competitive marketplaces.

Hardware Requirements

- Standard PC/Laptop (Intel i3/i5 or AMD Ryzen, 8–16 GB RAM)
- Internet-enabled device for web/mobile application access
- Optional: Mobile devices for farmer and buyer app interface

Software Requirements

- **Frontend:**
 - Web interface using HTML, CSS, JavaScript
 - Frameworks: React.js or Angular for dashboard and user interface
 - Mobile app interface (optional) for farmers/buyers
- **Backend:**
 - Python (Flask / Django) or Node.js for server-side logic
 - Database: MySQL / PostgreSQL / MongoDB to store contracts, produce data, and transactions
 - APIs for notifications (email/SMS) and price updates
- **Environment & Tools:**
 - Anaconda (if Python-based ML modules are included)

List Of Mini Projects

- VS Code / PyCharm for development
- OS: Windows/Linux

32)Project Title

Conversational Image Recognition Chatbot

Abstract

This project develops a **deep learning-based image recognition chatbot** that combines **computer vision** and **natural language processing (NLP)** to interact with users through images and text. Unlike traditional chatbots, this system can detect objects in images uploaded by users and generate contextually relevant responses. By integrating **object detection models** with NLP techniques, the chatbot ensures responses are both **semantically and grammatically accurate**. It enables meaningful **conversational interactions** based on visual inputs, offering a new dimension to human-AI communication. This solution is ideal for applications in **customer support, e-commerce, education, and accessibility tools**, providing an interactive, intelligent, and visually-aware conversational agent.

Hardware Requirements

- PC/Laptop with NVIDIA GPU (RTX series recommended for real-time inference)
- Minimum 16 GB RAM
- Webcam or image upload functionality

Software Requirements

- **Backend / Model Development:**
 - Python (3.8 or above)
 - Deep Learning Libraries: TensorFlow / PyTorch
 - Computer Vision Models: YOLOv8 / ResNet / EfficientNet for object detection
 - NLP Libraries: Hugging Face Transformers, spaCy, NLTK for language processing
 - Flask / Django for web server development

List Of Mini Projects

- **Frontend / User Interface:**

- HTML, CSS, JavaScript for dashboard and chat interface
- React.js / Angular for interactive frontend
- Image upload functionality integrated with chatbot interface

- **Database / Storage:**

- MySQL / MongoDB to store conversation logs, user queries, and detected image data

- **Environment & Tools:**

- Anaconda for Python environment management
- VS Code / Jupyter Notebook for model training and testing
- OS: Windows/Linux

33) Project Title

Software Solution to Trace the End Receiver of a Cryptocurrency Transaction

Abstract

This project develops a **rule-based solution** to trace Ethereum transactions and identify the **end receiver** in a transaction chain. Unlike machine learning approaches, the system leverages **heuristic rules** for a more interpretable and efficient tracing mechanism. By taking an Ethereum address as input, the system uses the **Etherscan API** to fetch all transactions related to the address. The transaction data is then stored in a **Neo4j graph database**, allowing visualization of transaction flows, intermediate addresses, and relationships between wallets. This tool helps **blockchain analysts, law enforcement agencies, and researchers** track illicit activities such as fraud and money laundering. The project demonstrates the effectiveness of heuristic-based analysis in simplifying blockchain tracing while reducing computational overhead and providing real-time insights into fund movements.

Hardware Requirements

- PC/Laptop with minimum 8–16 GB RAM

List Of Mini Projects

- Internet connection for API access
- Standard storage (50 GB recommended) for transaction logs and database

Software Requirements

- **Backend / Processing:**
 - Python (3.8 or above) for API interaction and data processing
 - Neo4j for graph database visualization and storage
 - Etherscan API integration for retrieving transaction data
 - Libraries: Requests, Pandas, NetworkX for graph manipulation
- **Frontend / Visualization:**
 - HTML, CSS, JavaScript for dashboard and interactive graphs
 - Optional: D3.js or Cytoscape.js for interactive network visualization
- **Environment & Tools:**
 - VS Code / PyCharm for development
 - Neo4j Desktop or Neo4j Aura cloud instance
 - Anaconda (optional) for managing Python environments
 - OS: Windows/Linux

34) Project Title

Policy Advisor: RAG-Based Personalized Insurance Recommendation System

Abstract

The Policy Advisor project focuses on developing an **RAG-based system** (Retrieval-Augmented Generation) to provide **personalized insurance recommendations** and guidance. The system leverages **web scraping** techniques to collect real-time data from multiple

List Of Mini Projects

insurance websites, including policy details, coverage options, and premium information. Using the retrieved data, the system generates user-specific advice, helping individuals make informed decisions about insurance policies. The combination of **retrieval-based data aggregation** and **AI-driven response generation** ensures accurate, up-to-date, and personalized recommendations for users seeking insurance-related assistance.

Hardware Requirements

- PC/Laptop with minimum 8 GB RAM
- Internet-enabled device for web scraping and user interface
- Optional: GPU (for faster AI model processing)

Software Requirements

- **Backend / AI & Data Processing:**
 - Python (3.8 or above)
 - Web scraping libraries: BeautifulSoup, Selenium, Scrapy
 - Retrieval-Augmented Generation (RAG) implementation: Hugging Face Transformers, PyTorch
 - Flask / Django for API and server-side development
 - Pandas / NumPy for data manipulation
- **Frontend / User Interface:**
 - HTML, CSS, JavaScript for web dashboard
 - React.js / Angular for interactive components
 - Optional: Bootstrap for responsive design
- **Database / Storage:**
 - MySQL / PostgreSQL / MongoDB to store scraped data, user queries, and recommendations

List Of Mini Projects

- **Environment & Tools:**

- Anaconda for Python environment management
- VS Code / Jupyter Notebook for development and testing
- OS: Windows/Linux

35) Project Title

Automated Redaction: Machine Learning-Based Sensitive Data Protection Tool

Abstract

"REDACT" is an advanced machine learning-based redaction system designed to **identify, obfuscate, and mask sensitive information** across multiple formats, including text files and documents. By leveraging **Natural Language Processing (NLP) and deep learning models**, the tool automatically detects **personally identifiable information (PII)**, ensuring compliance with privacy standards while preserving the contextual structure of the original data. Unlike traditional redaction methods, REDACT maintains **data usability**, enabling seamless integration with analytics, auditing, and reporting workflows. This system is particularly valuable for sectors like **healthcare, finance, and legal**, where data accuracy, privacy, and compliance are critical.

Hardware Requirements

- PC/Laptop with at least 8–16 GB RAM
- GPU (optional but recommended for faster model inference)
- Internet connection (for downloading models and libraries)

Software Requirements

- **Backend / NLP & Machine Learning:**
 - Python (3.8 or above)
 - Deep learning frameworks: TensorFlow or PyTorch
 - NLP libraries: Hugging Face Transformers, spaCy, NLTK
 - OCR libraries (if processing scanned documents): Tesseract OCR, OpenCV

List Of Mini Projects

- **Frontend / User Interface:**
 - HTML, CSS, JavaScript for dashboard
 - Optional: Streamlit / Flask for interactive web interface
- **Database / Storage:**
 - MySQL / MongoDB (for storing processed and original documents, if required)
- **Environment & Tools:**
 - Anaconda for managing Python environments
 - VS Code / Jupyter Notebook for development and testing
 - OS: Windows/Linux

36) Project Title

Emotion Recognizer: Hybrid CNN-Transformer Model for Audio-Based Emotion Detection

Abstract

Emotion recognition is critical for enhancing **human-computer interaction** in applications such as healthcare, customer support, and social robotics. This project proposes a **hybrid deep learning model** combining **Convolutional Neural Networks (CNN)** and **Transformer encoders** to recognize emotions from audio data. The CNN processes **mel spectrograms** of audio signals to extract spatial features, while the Transformer encoder captures **temporal dependencies** in pitch transitions to detect subtle emotional patterns. Data augmentation using **Additive White Gaussian Noise (AWGN)** improves model robustness to varying audio conditions. Trained and evaluated on the **RAVDESS dataset**, the system classifies audio clips into **eight emotional categories**, providing an effective framework for audio-based emotion recognition.

Hardware Requirements

- PC/Laptop with minimum 16 GB RAM
- GPU recommended (NVIDIA CUDA-enabled) for faster model training

List Of Mini Projects

- Internet connection for downloading datasets and pre-trained models

Software Requirements

- **Backend / Model Development:**
 - Python 3.8 or above
 - TensorFlow / PyTorch for deep learning
 - Librosa for audio processing
 - NumPy / Pandas for data handling
 - Matplotlib / Seaborn for visualizations
- **Frontend / Interface:**
 - Optional: Streamlit or Flask for audio input and real-time emotion prediction
 - HTML, CSS, JavaScript for dashboard (if web-based UI is implemented)
- **Environment & Tools:**
 - Anaconda for environment management
 - VS Code / Jupyter Notebook for development
 - OS: Windows/Linux

37) Project Title

Smart Parking System – ParkIt: IoT-Enabled Real-Time Parking Management

Abstract

The **Smart Parking System (ParkIt)** is an IoT-driven solution designed to optimize parking efficiency in urban and commercial spaces. By utilizing **IoT sensors, cloud computing, and real-time analytics**, the system provides accurate information on available parking spaces and guides users to the nearest spot. Integrated **mobile applications** enable users to **locate**,

List Of Mini Projects

reserve, and pay for parking, reducing traffic congestion, search time, and carbon emissions. The system also features **smart monitoring** for enhanced security and **automated management** for parking operators, aiming to improve parking utilization, urban mobility, and overall user experience.

Hardware Requirements

- IoT sensors (ultrasonic / infrared) to detect vehicle presence
- Microcontroller (Arduino, ESP8266, or Raspberry Pi)
- Wi-Fi / Internet connectivity
- Parking lot infrastructure for sensor placement

Software Requirements

- **Backend / IoT & Cloud:**
 - Python / Node.js for sensor data processing
 - Cloud platform (AWS IoT / Google Cloud / Azure IoT) for data storage and analytics
 - MQTT or HTTP protocols for IoT communication
- **Frontend / Mobile Application:**
 - Android Studio / Flutter / React Native for mobile app development
 - Google Maps API or OpenStreetMap for navigation
 - HTML, CSS, JavaScript for admin dashboard (optional)
- **Database / Storage:**
 - MySQL / Firebase / MongoDB for storing parking data, reservations, and user information
- **Environment & Tools:**
 - VS Code / PyCharm / Arduino IDE for development

List Of Mini Projects

- Testing platforms: Android Emulator or real devices
- OS: Windows / Linux / Mac

38) Project Title

2D to 3D Blueprint Transformation System

Abstract

The **2D to 3D Blueprint Transformation System** addresses the challenge of visualizing technical designs in three dimensions. This project automates the conversion of **2D blueprints** into **detailed 3D models**, enhancing design precision, spatial understanding, and project efficiency. By leveraging **Computer-Aided Design (CAD) tools** and **computational algorithms**, the system allows architects, engineers, and designers to simulate real-world objects, detect potential design errors early, and improve overall workflow. The project demonstrates practical applications across architecture, engineering, manufacturing, and gaming, streamlining design iterations, enhancing collaboration, and increasing accuracy in product development and construction.

Hardware Requirements

- PC/Laptop with high processing power (Intel i5/i7 or equivalent)
- Minimum 8GB RAM (16GB recommended for large models)
- Graphics card with OpenGL support for 3D rendering

Software Requirements

- **Backend / Processing:**
 - Python / C++ for algorithms and automation
 - OpenCV or similar libraries for blueprint processing
 - CAD software with scripting support (AutoCAD, SolidWorks, or Blender)
- **Frontend / Visualization:**

List Of Mini Projects

- Blender / Unity / Three.js for 3D model visualization
- Optional web-based dashboard: HTML, CSS, JavaScript
- **Environment & Tools:**
 - Anaconda or IDE like VS Code / PyCharm
 - OS: Windows / Linux / Mac

39) Project Title

Indian Sign Language (ISL) to Text Translation System

Abstract

The **Indian Sign Language (ISL) to Text Translation System** is designed to assist individuals with hearing or speech impairments by converting hand gestures into readable text in real-time. Using a webcam to capture live video of hand gestures, the system processes frames using **Python** and **computer vision techniques**. A **machine learning model** trained on a dataset of ISL gestures identifies and classifies hand signs into alphabets. Gestures are confirmed only after consistent detection for one second to ensure accuracy, and letters are combined to form words. This project addresses challenges such as variations in hand size, angles, and lighting conditions, providing a simple yet effective solution for real-time communication and inclusivity. Future enhancements may include additional gestures, multilingual support, and mobile integration.

Hardware Requirements

- PC or Laptop with USB/HD webcam
- Minimum 4GB RAM (8GB recommended)
- Graphics support for real-time video processing

Software Requirements

- **Backend / Processing:**

List Of Mini Projects

- Python 3.x
- OpenCV for video capture and image processing
- TensorFlow / Keras / PyTorch for gesture recognition model
- **Frontend / User Interface:**
 - HTML, CSS, and JavaScript for displaying recognized text
 - Optional GUI: Tkinter or PyQt
- **Database (Optional):**
 - CSV or JSON files to store gestures and mappings
- **Environment & Tools:**
 - Anaconda for Python environment management
 - IDE: VS Code / PyCharm
 - OS: Windows / Linux / Mac

40) Project Title

Innovative AI & AR-Based Toys and Games for Modern Users

Abstract

This project focuses on designing **interactive toys and games** that blend traditional physical play with **digital technologies** like **Augmented Reality (AR)** and **Artificial Intelligence (AI)**. Targeted toward the 18-30 age group, the system aims to enhance creativity, critical thinking, and engagement through immersive experiences. The proposed game includes AI-driven personalization, trivia quizzes, and real-time multiplayer features that allow users to compete globally or with friends. AR integration adds an interactive dimension where players can manipulate virtual objects to solve challenges. Gamification elements, such as rewards, badges, and achievements, motivate players while AI adjusts game difficulty and content dynamically based on performance. This innovative approach combines entertainment, learning, and technology, promoting creativity, inclusivity, and sustainability in modern toys and games.

List Of Mini Projects

Hardware Requirements

- PC/Laptop with camera (for AR interaction)
- Optional: Smartphone or tablet (for AR/AI mobile-based experiences)
- Microcontroller (for smart toys integration, e.g., Arduino / Raspberry Pi)

Software Requirements

- **Backend / Game Logic:**
 - Python or Node.js for AI/game logic
 - TensorFlow / PyTorch for AI personalization (adaptive difficulty, content recommendation)
- **Frontend / Interaction:**
 - Unity3D or Unreal Engine for AR game development
 - HTML, CSS, JavaScript (for web-based game dashboards)
 - Mobile AR SDKs: ARCore (Android) / ARKit (iOS)
- **Database:**
 - Firebase or MySQL to store user profiles, scores, achievements, and AI data
- **Development Tools:**
 - Visual Studio Code / PyCharm for coding
 - Blender / Maya (optional) for 3D models
 - Git for version control

41) Project Title

LLM-Based Prediction and Discovery of Advanced Magnetic Materials

List Of Mini Projects

Abstract

The project aims to leverage **Large Language Models (LLMs) and machine learning** to predict the properties of new compounds with potential as magnets or super magnets. Traditional material discovery processes are often **time-consuming and expensive**, but AI-powered approaches can accelerate the identification of materials with desired properties. By analyzing chemical structures, composition patterns, and historical experimental data, the system predicts magnetic properties, enabling faster and more efficient material development.

A key objective is to focus on **elements abundant in India**, reducing dependency on foreign imports and promoting self-reliance in critical industries. This approach not only streamlines material discovery but also contributes to **scientific understanding of material properties**, enabling targeted innovation for applications in electronics, renewable energy, and industrial machinery.

Hardware Requirements

- PC/Laptop with minimum 8GB RAM (16GB recommended)
- GPU-enabled system (NVIDIA GPU recommended) for faster model training
- Internet connectivity for dataset download and LLM model usage

Software Requirements

- **Programming & AI Frameworks:**
 - Python 3.8+
 - TensorFlow / PyTorch for ML modeling
 - Hugging Face Transformers library for LLM integration
- **Data Handling & Analysis:**
 - Pandas, NumPy for dataset processing
 - Scikit-learn for preprocessing and evaluation
 - Matplotlib / Seaborn for visualization
- **Database / Dataset:**

List Of Mini Projects

- Materials Project dataset or Open Quantum Materials Database (OQMD)
- Custom CSV/JSON datasets of material compositions and properties
- **Development Tools:**
 - Jupyter Notebook / VS Code for coding and experimentation
 - Git for version control

42)Project Title

DeepCrop: Deep Learning-Based Crop Price Prediction System

Abstract

Accurate crop price prediction is critical for farmers, traders, and policymakers to make informed decisions, manage risks, and ensure market stability. Agricultural markets are inherently complex and non-linear, influenced by factors such as weather conditions, crop yield, supply-demand fluctuations, and geopolitical events. Traditional statistical models often fail to capture these complexities.

DeepCrop is a deep learning-based system designed to predict future crop prices by analyzing historical price data, climate information, and market trends. The system employs a **deep neural network (DNN)** that learns patterns and dependencies within the data to provide accurate forecasts. Training and evaluation on real-world datasets demonstrate that the model adapts well to the intricacies of agricultural price fluctuations, outperforming traditional approaches.

This tool empowers stakeholders across the agricultural supply chain to optimize decisions, improve market planning, and reduce financial uncertainty.

Hardware Requirements

- PC/Laptop with minimum 8GB RAM (16GB recommended)
- GPU-enabled system (NVIDIA GPU recommended) for training deep learning models
- Internet connectivity for dataset download and libraries installation

List Of Mini Projects

Software Requirements

- **Programming & ML Frameworks:**
 - Python 3.8+
 - TensorFlow / PyTorch for building deep learning models
 - Keras for DNN implementation
- **Data Handling & Analysis:**
 - Pandas, NumPy for data processing
 - Scikit-learn for preprocessing and evaluation
 - Matplotlib / Seaborn for data visualization
- **Development Tools:**
 - Jupyter Notebook / VS Code
 - Git for version control
- **Datasets:**
 - Historical crop price data (government or Kaggle datasets)
 - Climate/weather datasets
 - Market trends and supply-demand data

43)Project Title

Next Word Prediction using BiLSTM

Abstract

List Of Mini Projects

Next word prediction is a key task in natural language processing that involves predicting the most probable next word in a sentence based on the preceding words. This project leverages a **Bidirectional Long Short-Term Memory (BiLSTM)** model, which reads sentences both forward and backward, allowing it to capture the complete context and semantic meaning more effectively than traditional methods like N-grams or standard LSTMs.

The model is trained using **word embeddings**, which convert words into numerical vectors that the model can process. BiLSTM's ability to understand context from both directions improves prediction accuracy, making it useful for applications such as **autocomplete systems, smart keyboards, and chatbots**. The research also addresses challenges like handling large vocabularies and maintaining fast prediction speeds for real-time applications.

Hardware Requirements

- PC/Laptop with at least 8GB RAM (16GB recommended)
 - GPU-enabled system (NVIDIA GPU recommended) for training deep learning models
 - Internet connection for downloading datasets and pre-trained embeddings
-

Software Requirements

- **Programming & ML Frameworks:**
 - Python 3.8+
 - TensorFlow / Keras or PyTorch for BiLSTM implementation
 - Gensim for word embeddings (optional)
- **Data Handling & Analysis:**
 - Pandas, NumPy for preprocessing and analysis
 - Scikit-learn for splitting datasets and evaluation
- **Visualization Tools:**
 - Matplotlib / Seaborn for plotting training performance

List Of Mini Projects

- **Development Environment:**
 - Jupyter Notebook / VS Code
- **Datasets:**
 - Large corpus of text data (e.g., Wikipedia dumps, open-source text datasets)

44) Project Title

AI-Powered Student Assistance Chatbot

Abstract

This project focuses on developing an **AI-powered chatbot** designed to assist students in accessing information about their college without the need for physical presence. The chatbot leverages **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** algorithms to interpret user queries and provide accurate responses related to examinations, admissions, academics, placements, and other college activities.

The system employs a **conversational user interface** that allows users to interact via text, enabling a smooth and user-friendly experience. By integrating the chatbot into a college website, students, staff, and even visitors can quickly find relevant information without navigating complex web pages. The chatbot analyzes the input, understands the context, and provides responses that mimic human-like interactions, enhancing accessibility and efficiency in information dissemination.

Keywords: Artificial Intelligence, Chatbot, Machine Learning, College Inquiry Chatbot, Student Assistance

Hardware Requirements

- PC/Laptop with minimum 8GB RAM (16GB recommended)
 - Internet connection for dataset access and API integrations
 - Optional GPU for faster training if using deep learning-based NLP models
-

List Of Mini Projects

Software Requirements

- **Programming & Frameworks:**
 - Python 3.8+
 - NLP libraries: NLTK, spaCy, or Hugging Face Transformers
 - Machine Learning frameworks: TensorFlow, PyTorch, or Scikit-learn
- **Web & Interface Development:**
 - HTML, CSS, JavaScript for front-end integration
 - Flask or Django for web deployment
- **Database:**
 - SQLite / MySQL for storing FAQ and student information
- **Development Environment:**
 - Jupyter Notebook / VS Code

45) Project Title

Crime Scene Analysis and Crime Prediction System

Abstract

This project focuses on **crime scene analysis** and the use of data-driven techniques for predicting crime-prone areas. It explores essential investigation methods, including crime scene management, documentation, evidence collection, and preservation, while emphasizing the role of technology and teamwork in the criminal justice system.

To enhance predictive capabilities, the project implements a **clustering approach** on historical crime data. Using the **K-Means algorithm**, crime records are classified and clustered to identify patterns and trends. This enables law enforcement agencies to **predict potential crime hotspots** and distinguish regions with a higher likelihood of criminal activity. Through case studies and analytical modeling, the project demonstrates how data-driven crime analysis can support effective investigation strategies and improve public safety.

List Of Mini Projects

Keywords: Crime Scene Analysis, K-Means Clustering, Crime Prediction, Criminal Justice, Data Analytics

Hardware Requirements

- PC/Laptop with minimum 8GB RAM (16GB recommended for large datasets)
 - Internet connection for accessing crime datasets
-

Software Requirements

- **Programming & Tools:**
 - Python 3.8+
 - Libraries: Pandas, NumPy, Matplotlib, Seaborn
 - Machine Learning: Scikit-learn (for K-Means clustering)
- **Database:**
 - CSV files or SQL database for storing crime data
- **Development Environment:**
 - Jupyter Notebook / VS Code

46) Project Title

PDF Analysis and Answer Generator Using Chatbot

Abstract

This project develops an **intelligent chatbot** capable of analyzing PDF documents and providing accurate answers to user queries. Using **Python Flask** as the backend, the system leverages **natural language processing (NLP)** with Gemini and **vector embeddings** for

List Of Mini Projects

semantic search. It can process various types of content, including text, tables, and images, and integrates **OCR (Optical Character Recognition)** to handle scanned documents.

The chatbot interprets user queries, extracts relevant information from PDFs, and generates context-aware responses in real-time. This project highlights the integration of AI-driven semantic understanding with a **user-friendly interface**, enabling seamless and interactive document querying for educational, corporate, and research purposes.

Keywords: PDF Analysis, Chatbot, Natural Language Processing, OCR, Semantic Search, Flask, Vector Embeddings

Hardware Requirements

- PC/Laptop with at least 8GB RAM
 - Internet connection for downloading libraries and dependencies
-

Software Requirements

- **Programming Language:** Python 3.8+
- **Frameworks & Libraries:**
 - Flask (for backend API and web server)
 - PyPDF2 / pdfplumber (for PDF parsing)
 - Gemini / SpaCy / Transformers (for NLP)
 - FAISS / Annoy (for vector embeddings and semantic search)
 - Tesseract OCR (for scanned PDF processing)
- **Development Environment:** Jupyter Notebook / VS Code

47) Project Title

List Of Mini Projects

LLM for Advanced Materials (Polymers)

Abstract

Polymers are critical materials across various industries, and understanding their properties is vital for innovation and application. This project develops a **web-based platform** that leverages **Machine Learning (ML)** and **Large Language Models (LLMs)** to predict and analyze polymer properties from their **SMILES (Simplified Molecular Input Line Entry System)** representations.

Users can input a polymer's SMILES notation to receive predictions of key properties such as **thermal stability, tensile strength, molecular weight**, and more using trained ML algorithms. Additionally, the system employs LLMs to provide **contextual insights**, including potential applications, environmental considerations, and alternative materials. For polymers that are expensive or unsuitable for specific applications, the system recommends **cost-effective and performance-optimized substitutes**, aiding researchers and industry professionals in decision-making.

By integrating **computational chemistry, predictive modeling, and natural language processing**, the platform streamlines material selection, fosters innovation, and promotes sustainable practices through environmentally friendly alternatives.

Keywords: Polymers, Machine Learning, Large Language Models, SMILES, Material Prediction, Sustainable Materials

Hardware Requirements

- PC/Laptop with 8GB+ RAM
 - Internet access for cloud-based ML model queries and LLM API access
-

Software Requirements

- **Programming Language:** Python 3.8+
- **Frameworks & Libraries:**
 - Flask or Django (for web application backend)

List Of Mini Projects

- Scikit-learn / PyTorch / TensorFlow (for ML models)
- RDKit (for handling SMILES and chemical features)
- Transformers / Hugging Face (for LLM integration)
- HTML, CSS, JavaScript (for frontend interface)
- **Development Environment:** VS Code / Jupyter Notebook

48) Project Title

Automated Student Attendance Using Deep Learning and Transformer Models for Object Counting

Abstract

Object counting in natural scenes is a complex task in computer vision and has critical applications in fields such as **classroom attendance, surveillance, retail, autonomous driving, and environmental monitoring**. Traditional methods often rely on detailed instance-level annotations like bounding boxes or pixel-level masks, which are **time-consuming and labor-intensive** to acquire.

This project proposes an **image-level supervised approach** for student attendance counting that minimizes annotation requirements by using only **global count information**. The system generates an **object category density map**, predicting both the **total count** and **spatial distribution** of students in a classroom. This approach enhances counting **accuracy in dense, occluded, or crowded scenarios**, making it scalable and practical for real-time attendance monitoring. By integrating **deep learning with Transformer-based models**, the project leverages advanced attention mechanisms for improved feature extraction and precise counting.

Hardware Requirements

- PC/Laptop with **GPU support (NVIDIA CUDA)** for deep learning training
- Minimum **8GB RAM**
- Camera or dataset of classroom images

List Of Mini Projects

Software Requirements

- **Programming Language:** Python 3.8+
- **Frameworks & Libraries:**
 - PyTorch / TensorFlow (for deep learning and Transformer models)
 - OpenCV (for image processing)
 - NumPy, Pandas, Matplotlib (for data handling and visualization)
- **Development Environment:** Jupyter Notebook / VS Code

49) Project Title

Time Series Anomaly Detection Using Transformers

Abstract

Unsupervised detection of anomalies in time series data is a challenging task, as it requires the model to identify points that deviate significantly from normal patterns without labeled data. Traditional approaches focus on **pointwise representations** or **pairwise associations**, but these methods often struggle to capture the complex temporal dynamics of real-world time series.

This project leverages **Transformers** to unify pointwise and pairwise modeling by using **self-attention weights** to capture the association of each time point with the entire series. The key insight is that **anomalous points**, being rare, have limited associations with distant points and primarily associate with **adjacent time points**. This **adjacent-concentration bias** allows anomalies to be distinguished from normal points using an **association-based criterion**, termed **Association Discrepancy**.

We implement the **Anomaly Transformer**, featuring a novel **Anomaly-Attention mechanism** to compute the association discrepancy. A **minimax strategy** is incorporated to enhance the distinguishability between normal and abnormal points, improving detection performance. This

List Of Mini Projects

method provides an effective, unsupervised approach to detecting anomalies in time series data, applicable to domains such as finance, IoT, healthcare, and industrial monitoring.

Hardware Requirements

- PC/Laptop with GPU support (NVIDIA CUDA recommended)
 - Minimum 8GB RAM
 - Storage for datasets
-

Software Requirements

- **Programming Language:** Python 3.8+
- **Frameworks & Libraries:**
 - PyTorch / TensorFlow (for Transformer model implementation)
 - NumPy, Pandas (data handling)
 - Matplotlib / Seaborn (visualization of anomalies)
- **Development Environment:** Jupyter Notebook / VS Code

50) Project Title

AI-Based Crop Disease Detection Using CNN

Abstract

Agricultural productivity is often affected by crop diseases, leading to significant economic losses. Early detection of diseases is crucial for sustainable farming and food security. This project develops an AI-based system using Convolutional Neural Networks (CNN) to automatically detect and classify crop diseases from leaf images. The model extracts features such as color, texture, and shape patterns from the images to accurately identify disease types. Users can upload images via a web or mobile interface, and the system provides real-time predictions along with preventive or corrective measures. This approach reduces reliance on

List Of Mini Projects

manual inspection and expert knowledge, enabling timely interventions and improved crop management.

Hardware Requirements

- PC/Laptop with GPU support (NVIDIA CUDA recommended)
- Minimum 8GB RAM
- Storage for image datasets

Software Requirements

- Programming Language: Python 3.8+
- Frameworks & Libraries:
 - TensorFlow / PyTorch (for CNN model)
 - OpenCV (image preprocessing)
 - NumPy, Pandas (data handling)
 - Matplotlib / Seaborn (visualization)
- Development Environment: Jupyter Notebook / VS Code
- Optional: Flask / Streamlit for web interface

List Of Mini Projects

Project 51: AI-Based Traffic Management System

Abstract

Urban traffic congestion leads to delays, fuel consumption, and pollution. This project develops an AI-based traffic management system that dynamically optimizes traffic light timings using real-time data from IoT sensors and cameras. The AI engine prioritizes traffic flow from congested directions, reducing waiting times and improving efficiency.

Hardware Requirements

- PC/Laptop with GPU
- IoT sensors / Camera feeds (for simulation)
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, TensorFlow / PyTorch
- OpenCV, NumPy, Pandas
- Flask / Django

Front-End

- Web dashboard to display live traffic status
- Technologies: HTML, CSS, JavaScript, optional React.js

Back-End

- Flask/Django API for processing real-time traffic data

List Of Mini Projects

- ML model integration for traffic prediction
 - Database: MySQL / PostgreSQL for storing historical traffic data
 - Dynamic traffic signal adjustment
 - Reduced congestion and travel time
 - Real-time monitoring dashboard for city traffic management
-

Project 52: AI-Powered Student Assistance Chatbot

Abstract

This AI chatbot assists students by answering queries about admissions, exams, placements, and college activities. It uses NLP to understand questions and provide accurate responses, improving website usability and student engagement.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, NLTK / spaCy / GPT-based APIs
- Flask for backend
- HTML, CSS, JavaScript for front-end

Front-End

- Chat interface embedded in the college website
- Technologies: HTML, CSS, JavaScript, optional React.js

Back-End

List Of Mini Projects

- Flask API to handle queries
 - Integration with NLP model for intent recognition
 - Database: SQLite/PostgreSQL for storing FAQs and chat logs
-

Project 53: Predicting Crop Prices Using Deep Learning

Abstract

DeepCrop is a deep learning system that predicts crop prices using historical market data, weather conditions, and supply-demand trends. Accurate predictions help farmers, traders, and policymakers make informed decisions.

Hardware Requirements

- PC/Laptop with GPU
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, TensorFlow / Keras
- NumPy, Pandas, Matplotlib

Front-End

- Web dashboard for inputting crop details and viewing price predictions
- Technologies: HTML, CSS, JavaScript, optional React.js

Back-End

- Flask API to run trained deep learning model
 - Database: MySQL/PostgreSQL to store historical crop prices and trends
-

List Of Mini Projects

Project 54: Smart Parking System

Abstract

The Smart Parking System uses IoT sensors and cloud analytics to guide drivers to available parking spots in real-time, reduce congestion, and optimize space usage in urban areas.

Hardware Requirements

- PC/Laptop
- IoT sensors / simulated data for parking availability
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, Flask / Django
- NumPy, Pandas, OpenCV (optional)
- Database: MySQL/PostgreSQL

Front-End

- Web/mobile dashboard for drivers to locate and reserve parking
- Technologies: HTML, CSS, JavaScript, optional React.js

Back-End

- Flask API to process parking availability data
- Integration with database for reservations and history

Project 55: Automated Student Attendance Using Deep Learning

Abstract

This system uses computer vision and Transformer models to automate attendance by

List Of Mini Projects

counting students in classrooms from camera feeds. It improves efficiency and accuracy compared to manual methods.

Hardware Requirements

- PC/Laptop with GPU
- Minimum 8GB RAM
- Camera for live capture

Software Requirements

- Python 3.8+, OpenCV, PyTorch / TensorFlow
- Flask / Django for API integration

Front-End

- Web dashboard for teachers to monitor attendance
- Technologies: HTML, CSS, JavaScript, optional React.js

Back-End

- Flask API receives video frames, runs object counting model, and logs attendance
- Database: MySQL/PostgreSQL to store attendance records

Project 56: PDF Analysis and Answer Generator Using Chatbot

Abstract

This project creates an intelligent chatbot that analyzes PDF documents and answers user queries. Using NLP and vector embeddings, the system extracts context-aware information from text, tables, and images, including scanned PDFs through OCR. It provides real-time interactive responses, improving accessibility to document content.

Hardware Requirements

- PC/Laptop

List Of Mini Projects

- Minimum 8GB RAM

Software Requirements

- Python 3.8+, Flask
- NLP libraries: Gemini, spaCy, or BERT
- OCR: Tesseract
- Database: SQLite / PostgreSQL

Front-End

- Web chat interface for uploading PDFs and querying
- Technologies: HTML, CSS, JavaScript

Back-End

- Flask API to process PDF uploads and handle queries
 - Vector embeddings for semantic search
 - NLP model integration for answering questions
-

Project 57: Indian Sign Language to Text Translation

Abstract

This system translates Indian Sign Language (ISL) gestures into text in real-time. Using a webcam, it captures hand gestures and identifies letters using a trained ML model. The solution improves communication for individuals with hearing or speech difficulties.

Hardware Requirements

- PC/Laptop with webcam
- Minimum 8GB RAM

Software Requirements

List Of Mini Projects

- Python 3.8+, OpenCV
- Machine Learning libraries: TensorFlow / PyTorch
- Front-end: HTML, CSS, JavaScript

Front-End

- Web interface for live video capture
- Displays translated text in real-time

Back-End

- Python Flask API to process video frames
- ML model classifies gestures into alphabets
- Combines letters to form words

Project 58: Conceptualize and Develop Unique Toys & Games

Abstract

This project encourages developing interactive toys and games using AI and Augmented Reality (AR). Targeting ages 18–30, it combines physical play with digital elements for engagement, creativity, and learning. The system tracks user interactions and adapts difficulty levels using AI personalization.

Hardware Requirements

- PC/Laptop
- Mobile device (optional for AR)
- Minimum 8GB RAM

Software Requirements

- Python / Unity3D

List Of Mini Projects

- AR SDK (ARCore / ARKit)
- ML libraries: TensorFlow / PyTorch

Front-End

- Mobile or web interface with AR features
- Game interaction dashboard

Back-End

- Python / Unity server for AI personalization
- Stores user performance and progress
- Database: SQLite / PostgreSQL

Project 59: Retail Finance Advisor Using LLM

Abstract

A web-based system leveraging LLMs to provide personalized financial advice to retail store owners. The system analyzes sales, expenses, and tax data using Retrieval-Augmented Generation (RAG) to generate context-aware financial insights.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, HuggingFace Transformers / OpenAI API
- Flask / Django for web framework
- Database: MySQL / PostgreSQL

List Of Mini Projects

Front-End

- Web dashboard to input financial data and view advice
- Technologies: HTML, CSS, JavaScript

Back-End

- Flask API to handle data input and generate financial advice
 - Integration with LLM for personalized suggestions
 - Database stores historical financial data
-

Project 60: Automated Redaction System (REDACT)

Abstract

This project develops a machine learning-based system to automatically redact sensitive information (PII) from documents while maintaining structure. It uses NLP and deep learning for accurate, efficient, and scalable data protection across industries like healthcare, finance, and legal sectors.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, TensorFlow / PyTorch
- NLP libraries: spaCy, NLTK
- Flask / Django for web interface
- Database: SQLite / PostgreSQL

Front-End

List Of Mini Projects

- Web interface to upload and download redacted files
- Technologies: HTML, CSS, JavaScript

Back-End

- Flask API for document processing
- NLP and deep learning model identifies PII for redaction
- Database stores metadata and audit logs

Project 61: Real-Time Log Monitoring and Alert System

Abstract

This project builds a real-time log monitoring system that collects logs from servers, applications, and databases, analyzing them for errors, anomalies, and performance issues. It uses a combination of ELK stack (Elasticsearch, Logstash, Kibana) and alerting mechanisms to notify admins about critical events in real-time.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python / Java
- ELK Stack (Elasticsearch, Logstash, Kibana)
- Slack / Email for alerts

Front-End

- Dashboard using Kibana for visualizing logs and analytics

List Of Mini Projects

Back-End

- Log collection via Logstash
 - Elasticsearch for storing and querying logs
 - Alerting service for anomalies
-

Project 62: DevOps CI/CD Pipeline Automation

Abstract

This project automates the Continuous Integration and Continuous Deployment (CI/CD) process using Jenkins, Docker, and Kubernetes. It ensures faster, reliable, and automated deployment of applications with version control integration and automated testing.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Jenkins, Docker, Kubernetes
- Git / GitHub / GitLab
- Python / Node.js

Front-End

- Web dashboard in Jenkins for pipeline status and logs

Back-End

- Jenkins jobs automate build, test, and deployment
- Docker containers package the application

List Of Mini Projects

- Kubernetes manages deployment and scaling

Project 63: Big Data Sentiment Analysis on Twitter

Abstract

This project analyzes Twitter data in real-time using big data technologies to understand public sentiment about products, brands, or events. It leverages Apache Spark for data processing and NLP models for sentiment classification.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python, Apache Spark
- Tweepy (Twitter API)
- NLTK / TextBlob / Transformers

Front-End

- Web dashboard to visualize sentiment trends
- Technologies: HTML, CSS, JavaScript, Plotly

Back-End

- Spark processes real-time tweets
- Sentiment classification using NLP models
- Store processed results in MongoDB / MySQL

List Of Mini Projects

Project 64: Automated Infrastructure Provisioning using Terraform

Abstract

This project automates cloud infrastructure setup using Terraform, allowing developers to define, deploy, and manage servers, networks, and storage with code. It ensures consistent and repeatable deployments across environments.

Hardware Requirements

- PC/Laptop
- Cloud account (AWS / Azure / GCP)

Software Requirements

- Terraform, AWS CLI / Azure CLI / GCP SDK
- Python / Bash for scripts

Front-End

- Web interface (optional) to trigger Terraform scripts

Back-End

- Terraform scripts define infrastructure
- Automate provisioning using CLI or pipelines

Project 65: Real-Time Vehicle Number Plate Recognition

Abstract

This project detects and recognizes vehicle license plates from live CCTV feeds using deep learning. It aids in security, traffic management, and parking monitoring.

Hardware Requirements

List Of Mini Projects

- PC/Laptop with GPU
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, OpenCV
- TensorFlow / PyTorch
- Tesseract OCR

Front-End

- Web interface for live camera feed and plate display

Back-End

- CNN / YOLO model detects vehicles
- OCR reads number plate characters
- Database stores detected vehicles and timestamps

Project 66: IoT-Based Smart Home Automation with Big Data Analytics

Abstract

This project creates a smart home system using IoT devices and sensors, integrated with big data analytics to optimize energy usage and home security.

Hardware Requirements

- Raspberry Pi / Arduino
- Sensors: Temperature, Humidity, Motion
- PC/Laptop for analytics

List Of Mini Projects

Software Requirements

- Python 3.8+
- MQTT protocol, Node-RED
- Big data tools: Apache Spark / Hadoop

Front-End

- Mobile / Web app to monitor and control devices

Back-End

- Collect sensor data via MQTT
- Process and analyze data using Spark
- Trigger automated actions (lights, AC, alarms)

Project 67: Big Data Movie Recommendation System

Abstract

A recommendation system that uses big data and collaborative filtering to suggest movies to users based on viewing history and preferences. It handles large datasets efficiently using Apache Spark.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python / Scala

List Of Mini Projects

- Apache Spark, PySpark
- Database: MongoDB / PostgreSQL

Front-End

- Web dashboard for movie recommendations and ratings

Back-End

- Spark MLlib for collaborative filtering
- Store user and movie data in database
- Generate personalized recommendations

Project 68: DevOps Monitoring Dashboard with Prometheus & Grafana

Abstract

This project builds a monitoring and alerting system for DevOps using Prometheus and Grafana. It tracks metrics of servers, applications, and containers, providing insights for performance optimization.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Prometheus, Grafana
- Docker / Kubernetes for deployment
- Python / Bash

Front-End

List Of Mini Projects

- Grafana dashboard for real-time metric visualization

Back-End

- Prometheus collects metrics
 - Alerting rules configured for anomalies
 - Integrate with Slack/email for notifications
-

Project 69: Big Data Fraud Detection System

Abstract

A system that detects fraudulent transactions in real-time using big data and machine learning. It analyzes transactional data streams and identifies anomalies indicative of fraud.

Hardware Requirements

- PC/Laptop with GPU
- Minimum 8GB RAM

Software Requirements

- Python, Apache Spark
- ML libraries: Scikit-learn, PyTorch

Front-End

- Web dashboard for alerts and transaction visualization

Back-End

- Spark streaming for real-time data processing
- ML model detects anomalies and flags transactions

List Of Mini Projects

- Database stores flagged transactions

Project 70: Kubernetes Cluster Auto-Scaling with Big Data Metrics

Abstract

This project implements auto-scaling of Kubernetes clusters based on real-time application load and big data metrics. It ensures efficient resource utilization and cost optimization.

Hardware Requirements

- PC/Laptop
- Cloud environment (AWS / GCP / Azure)

Software Requirements

- Kubernetes, Docker
- Prometheus for metrics
- Python / Bash

Front-End

- Dashboard to monitor cluster status and scaling events

Back-End

- Prometheus collects CPU, memory, and load metrics
- Custom controller triggers scaling events
- Kubernetes handles pod scaling automatically

Project 71: Real-Time Video Analytics for Smart Surveillance

List Of Mini Projects

Abstract

This project implements real-time video analytics using deep learning to detect unusual activities, unauthorized access, or safety violations in public spaces or workplaces. It leverages YOLOv8 for object detection and OpenCV for video streaming, enabling automated alerts to security personnel.

Hardware Requirements

- PC/Laptop with GPU
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, OpenCV, PyTorch
- Flask for dashboard
- MongoDB for storing alerts

Front-End

- Web dashboard displaying live video feed and detected alerts

Back-End

- Deep learning model detects objects and unusual behavior
- Alert system sends notifications via email or SMS
- Database logs incidents with timestamps

Project 72: AI-Powered Code Review Assistant

Abstract

This project develops an AI assistant to automatically review code for style compliance, bug detection, and optimization suggestions. It uses LLMs like GPT or CodeBERT to analyze code syntax and semantics.

Hardware Requirements

List Of Mini Projects

- PC/Laptop with 8GB RAM
- Internet connection for API access

Software Requirements

- Python 3.8+, VS Code
- OpenAI API / Hugging Face Transformers
- Git for code integration

Front-End

- Web interface for developers to upload code snippets

Back-End

- LLM analyzes code for errors, best practices, and optimization suggestions
 - Stores review results in a database
 -
-

Project 73: IoT-Based Smart Agriculture with Predictive Analytics

Abstract

This project integrates IoT sensors with big data analytics to monitor crop health, soil moisture, and weather conditions. Predictive models forecast irrigation needs and crop yields, improving farm efficiency.

Hardware Requirements

- Raspberry Pi / Arduino
- Sensors: Moisture, Temperature, Humidity
- PC/Laptop for analytics

Software Requirements

List Of Mini Projects

- Python 3.8+, MQTT, Node-RED
- Apache Spark for analytics
- Dash / Plotly for dashboard

Front-End

- Web/mobile dashboard for farmers to monitor field data

Back-End

- IoT devices send real-time data
- Spark processes data and runs predictive models
- Alerts sent for irrigation and pest management

Project 74: AI-Driven Predictive DevOps Analytics

Abstract

This project leverages AI and big data to predict potential failures, performance degradation, or deployment risks in DevOps pipelines. It uses historical build, deployment, and system metrics for forecasting.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python, Apache Spark, Scikit-learn
- Prometheus and Grafana for metrics collection

List Of Mini Projects

Front-End

- Grafana dashboard showing predicted risks and alerts

Back-End

- Historical metrics processed in Spark
 - ML model predicts future failures and bottlenecks
 - Alerting system triggers preventive measures
-

Project 75: Big Data-Enabled E-commerce Recommendation System

Abstract

This project develops a recommendation engine for e-commerce platforms using big data and collaborative filtering. It predicts user preferences and enhances shopping experience by suggesting products based on behavior and trends.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python, Apache Spark, PySpark MLlib
- MongoDB / PostgreSQL
- Dash / React.js for dashboard

Front-End

- Web interface showing personalized product recommendations

Back-End

List Of Mini Projects

- Spark processes large-scale user and product data
 - Collaborative filtering and ML models generate recommendations
 - Database stores results and user interactions
-

Project 76: Cloud Cost Optimization using Predictive Analytics

Abstract

This project analyzes cloud resource usage and predicts future consumption trends using big data. It provides recommendations to optimize costs and resource allocation for cloud infrastructure.

Hardware Requirements

- PC/Laptop
- Cloud account (AWS / Azure / GCP)

Software Requirements

- Python, Apache Spark, Pandas
- Cloud APIs for resource usage

Front-End

- Web dashboard showing predicted usage and cost-saving recommendations

Back-End

- Collect cloud usage metrics
 - Predict future demand using ML models
 - Suggest optimal resource allocation
-

List Of Mini Projects

Project 77: AI-Powered Resume Screening System

Abstract

This project automates recruitment by analyzing resumes using NLP and ML to match candidates with job requirements. It scores candidates based on skills, experience, and qualifications.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python, NLTK / SpaCy, Scikit-learn
- Flask for web interface

Front-End

- Web interface for HR managers to upload resumes and view candidate scores

Back-End

- NLP extracts skills and experience from resumes
- ML model matches candidates with job descriptions
- Database stores scores and candidate details

Project 78: Real-Time Network Intrusion Detection System (NIDS)

Abstract

This project builds a real-time NIDS using big data and ML to detect cyber-attacks, anomalies, and unauthorized access in network traffic.

Hardware Requirements

List Of Mini Projects

- PC/Laptop with network adapter
- Minimum 8GB RAM

Software Requirements

- Python, Scapy, Apache Spark
- ML libraries (Scikit-learn / PyTorch)

Front-End

- Web dashboard showing network alerts, attack types, and threat levels

Back-End

- Real-time packet capture and preprocessing
- ML model classifies normal vs malicious traffic
- Alerting system triggers notifications

Project 79: Smart Energy Management System Using Big Data

Abstract

This project analyzes energy consumption patterns using IoT sensors and big data analytics to optimize usage and reduce costs in smart buildings.

Hardware Requirements

- IoT devices / Raspberry Pi
- PC/Laptop for analytics

Software Requirements

- Python 3.8+, MQTT, Apache Spark

List Of Mini Projects

- Dashboard: Dash / Plotly

Front-End

- Web/mobile dashboard for energy monitoring and control

Back-End

- Collect real-time consumption data
 - Analyze trends and detect inefficiencies
 - Provide actionable recommendations for energy optimization
-

Project 80: AI-Powered Chatbot for IT Support with Big Data Integration

Abstract

This project develops an AI chatbot that integrates big data logs and analytics to provide IT support, troubleshooting, and automated ticketing.

Hardware Requirements

- PC/Laptop
- Minimum 8GB RAM

Software Requirements

- Python 3.8+, Flask, NLP (Transformers)
- Database: MongoDB / MySQL

Front-End

- Web interface for users to interact with IT chatbot

Back-End

List Of Mini Projects

- Chatbot uses NLP to understand queries
- Integrates with system logs and analytics to provide solutions
- Automates ticket creation and issue tracking

81. Real-Time Social Media Sentiment Analysis

Abstract:

This project analyzes real-time social media data (e.g., Twitter) to determine public sentiment about events, products, or trends. It uses APIs to collect streaming data and natural language processing (NLP) techniques to classify text as positive, negative, or neutral. The system can visualize trends over time and provide dashboards for decision-makers.

Hardware Requirements:

- PC/Laptop with minimum 8GB RAM
- Internet connection for streaming data

Software Requirements:

- Python 3.8+
- Libraries: Tweepy, NLTK, TextBlob, Pandas, Matplotlib
- Flask/Django for dashboard (Back-End)
- React or HTML/CSS/JavaScript (Front-End)

Front-End & Back-End Design:

- Back-End: Flask/Django handles API integration, NLP processing, and stores sentiment results in a database (SQLite/MySQL).
- Front-End: Dashboard displays real-time sentiment charts, trending topics, and filters for hashtags or keywords.

List Of Mini Projects

82. Project

Movie Recommendation System with Big Data

Abstract:

This project builds a personalized movie recommendation system using collaborative filtering and big data analytics. Using user ratings and movie metadata, the system predicts what movies a user might like. The system leverages Spark for large datasets to ensure scalability.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for datasets

Software Requirements:

- Python 3.8+
- Apache Spark
- Pandas, NumPy, Scikit-learn
- Flask/Django (Back-End)
- HTML/CSS/React (Front-End)

Front-End & Back-End Design:

- **Back-End:** Spark handles data processing, collaborative filtering, and stores recommendations in a database.
- **Front-End:** Web interface allows users to rate movies, view recommended movies, and search movies.

List Of Mini Projects

83.Project

Traffic Flow Prediction Using Big Data

Abstract:

Predict traffic congestion in urban areas using historical and real-time traffic data. The system uses machine learning models to forecast traffic flow, enabling optimized routing and congestion management.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for traffic datasets

Software Requirements:

- Python 3.8+
- Apache Spark
- Pandas, Scikit-learn, Matplotlib
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Collects traffic data, trains ML models, and predicts congestion.
 - Front-End: Displays real-time maps with predicted traffic congestion and suggested routes.
-

List Of Mini Projects

84.Project

Retail Customer Segmentation Using K-Means Clustering

Abstract:

Segment retail customers based on purchasing patterns to support targeted marketing campaigns. The system clusters customers using K-Means, analyzing demographics, purchase frequency, and transaction value.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for retail datasets

Software Requirements:

- Python 3.8+
- Pandas, NumPy, Matplotlib, Scikit-learn
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Clustering algorithms process customer data and generate segments.
 - Front-End: Visual dashboard shows clusters, key metrics, and customer profiles.
-

85.Project

Fraud Detection in Banking Transactions

List Of Mini Projects

Abstract:

Detect fraudulent banking transactions using machine learning models trained on historical transaction data. The system identifies abnormal patterns in real-time and triggers alerts.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

- Python 3.8+
- Pandas, NumPy, Scikit-learn
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: ML models analyze transaction data and classify as normal or fraudulent.
- Front-End: Dashboard displays flagged transactions, alerts, and analytics.

86.Project

IoT-Based Smart Home Automation System

Abstract:

This project develops an IoT-driven smart home system that allows users to control lights, fans, and appliances remotely via a web or mobile app. It integrates sensors to monitor temperature, humidity, and motion, providing automation and energy efficiency.

Hardware Requirements:

- Raspberry Pi / Arduino
- Sensors: Temperature, Humidity, Motion

List Of Mini Projects

- Relay modules for appliances
- PC/Laptop with internet connection

Software Requirements:

- Python 3.8+
- Flask/Django (Back-End)
- MQTT protocol for IoT communication
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Flask server receives IoT sensor data, processes commands, and stores device status in a database.
- Front-End: Dashboard displays real-time sensor readings and allows control of home appliances.

87.Project

DevOps CI/CD Pipeline Automation

Abstract:

This project implements a continuous integration and continuous deployment (CI/CD) pipeline for a web application using DevOps tools. It automates build, test, and deployment stages to improve software delivery efficiency.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Docker support

Software Requirements:

List Of Mini Projects

- Git, Jenkins
- Docker, Kubernetes
- Python / Node.js (application)
- GitHub / GitLab repository

Front-End & Back-End Design:

- Back-End: Jenkins handles automated builds, tests, and deployments to Docker containers or Kubernetes clusters.
- Front-End: Web app shows deployment status and logs for monitoring.

88.Project

Big Data Analytics for E-Commerce Trends

Abstract:

Analyze large-scale e-commerce data to understand customer behavior, product trends, and sales patterns. The system uses Hadoop/Spark for processing big datasets and generates actionable insights.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for large datasets

Software Requirements:

- Apache Spark / Hadoop
- Python 3.8+, PySpark
- Pandas, Matplotlib, Seaborn

List Of Mini Projects

- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Spark processes transactional datasets to identify top-selling products, peak hours, and customer segmentation.
 - Front-End: Dashboard visualizes analytics using charts, graphs, and reports.
-

89.Project

Cloud-Based Log Monitoring System

Abstract:

This project develops a cloud-based system to monitor server logs in real-time, detect anomalies, and generate alerts for system administrators. It improves incident management and reduces downtime.

Hardware Requirements:

- PC/Laptop with internet connection
- Cloud account (AWS / Azure / GCP)

Software Requirements:

- Python 3.8+
- ELK Stack (Elasticsearch, Logstash, Kibana)
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

List Of Mini Projects

- **Back-End:** Collects logs from servers, indexes them in Elasticsearch, and sets up anomaly detection.
 - **Front-End:** Dashboard visualizes log trends, anomalies, and provides alert notifications.
-

90.Project

AI-Powered Resume Screening System

Abstract:

This project automates the recruitment process by screening resumes using AI. It extracts relevant information like skills, experience, and education, and ranks candidates based on job requirements.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

- Python 3.8+
- NLP Libraries: Spacy, NLTK, PyPDF2
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Extracts text from resumes, parses key information, and scores candidates based on matching job criteria.
- **Front-End:** Dashboard shows ranked candidates, search functionality, and detailed profiles.

91.Project

List Of Mini Projects

Real-Time Twitter Sentiment Analysis

Abstract:

This project analyzes live Twitter feeds to determine public sentiment about a product, service, or event. Using NLP and big data streaming frameworks, it provides real-time insights for businesses and marketers.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Internet connection

Software Requirements:

- Python 3.8+, Tweepy (Twitter API)
- PySpark / Apache Kafka
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Streams live tweets, preprocesses text, applies sentiment classification, stores results in a database.
 - Front-End: Displays live sentiment trends using charts and graphs.
-

92.Project

Smart Traffic Management System Using Big Data

Abstract:

This project collects traffic sensor data from cameras and IoT devices, analyzes congestion patterns, and provides dynamic traffic control recommendations using AI and big data analytics.

Hardware Requirements:

List Of Mini Projects

- PC/Laptop with 8GB+ RAM
- Traffic sensor datasets or simulated data

Software Requirements:

- Python 3.8+
- Apache Spark / Hadoop
- TensorFlow / PyTorch (AI model)
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Aggregates traffic data, predicts congestion levels, and provides traffic signal recommendations.
 - **Front-End:** Displays real-time traffic maps, congestion alerts, and route suggestions.
-

93.Project

AI-Based Cloud Resource Optimizer

Abstract:

This project develops an AI system that monitors cloud resources (CPU, memory, storage) and optimizes allocation using predictive analytics to reduce costs and improve performance.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Cloud account (AWS / Azure / GCP)

List Of Mini Projects

Software Requirements:

- Python 3.8+
- Pandas, NumPy
- TensorFlow / PyTorch
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Collects resource usage metrics, predicts future usage, adjusts allocation dynamically.
 - **Front-End:** Displays resource usage, alerts, and optimization recommendations.
-

94.Project

Big Data-Based Healthcare Analytics

Abstract:

This project analyzes large-scale healthcare datasets to predict disease outbreaks, patient risk factors, and treatment outcomes using AI and big data analytics.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for healthcare datasets

Software Requirements:

- Python 3.8+, Pandas, NumPy
- PySpark / Hadoop

List Of Mini Projects

- Scikit-learn / TensorFlow
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Processes patient data, applies predictive models for disease risk, and stores insights in a database.
 - Front-End: Visualizes patient risk, outbreak predictions, and trends.
-

95.Project

AI-Based DevOps Log Analyzer

Abstract:

This project automatically analyzes server logs to detect anomalies, predict failures, and suggest preventive measures using AI and big data analytics.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Cloud server for logs

Software Requirements:

- Python 3.8+
- PySpark / Hadoop
- TensorFlow / PyTorch
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

List Of Mini Projects

Front-End & Back-End Design:

- **Back-End:** Collects logs from servers, applies anomaly detection models, generates alerts.
 - **Front-End:** Dashboard shows log trends, detected anomalies, and predictions.
-

96.Project

Real-Time Stock Market Prediction

Abstract:

This project predicts stock price trends in real-time using historical and live market data with deep learning and big data analytics.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

- Python 3.8+, Pandas, NumPy
- PySpark / Hadoop
- TensorFlow / PyTorch (LSTM/GRU)
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Collects historical and real-time stock data, applies LSTM models for trend prediction.
- **Front-End:** Displays live stock trends, predictions, and alerts.

List Of Mini Projects

97.Project

Big Data-Based Fraud Detection System

Abstract:

This project detects fraudulent transactions in banking and finance using large-scale datasets and AI algorithms. It improves financial security and reduces losses.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM
- Storage for transactional datasets

Software Requirements:

- Python 3.8+, Pandas, NumPy
- PySpark / Hadoop
- TensorFlow / PyTorch (Classification)
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Processes transaction data, applies classification models, flags suspicious transactions.
- **Front-End:** Dashboard displays flagged transactions, fraud patterns, and analytics.

List Of Mini Projects

98.Project

AI-Powered E-Learning Analytics

Abstract:

This project analyzes student interaction and performance data in e-learning platforms to provide insights, predict learning outcomes, and recommend personalized learning paths.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

- Python 3.8+, Pandas, NumPy
- PySpark / Hadoop
- Scikit-learn / TensorFlow
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- Back-End: Processes learning data, predicts performance using ML models, generates insights and recommendations.
- Front-End: Dashboard visualizes student progress, performance trends, and personalized suggestions.

99.Project

AI-Powered Chatbot for DevOps Assistance

List Of Mini Projects

Abstract:

This project develops an intelligent chatbot to assist DevOps teams by providing solutions, scripts, and suggestions for CI/CD, monitoring, and cloud deployments.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

- Python 3.8+, NLTK, SpaCy
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)
- Docker / Kubernetes (Integration for testing)

Front-End & Back-End Design:

- Back-End: NLP-based chatbot interprets queries and fetches solutions from DevOps knowledge base.
 - Front-End: Chat interface for user queries and responses.
-

100.Project

Big Data-Based Energy Consumption Optimizer

Abstract:

This project analyzes energy consumption data from smart meters using big data techniques and AI to predict peak loads and suggest optimized usage strategies.

Hardware Requirements:

- PC/Laptop with 8GB+ RAM

Software Requirements:

List Of Mini Projects

- Python 3.8+, Pandas, NumPy
- PySpark / Hadoop
- TensorFlow / PyTorch
- Flask/Django (Back-End)
- React/HTML/CSS (Front-End)

Front-End & Back-End Design:

- **Back-End:** Collects and processes energy consumption data, predicts peak load, and provides usage recommendations.
- **Front-End:** Dashboard visualizes consumption trends, peak hours, and optimization strategies.